

Financial Services Track -- Prompt Library

All prompts are copy-paste ready. Each has a primary version and a fallback.

Step 1: Database Setup

1.1 Primary Prompt

Please execute the setup script from the GitHub repository we connected. Run this file to create the financial services database, warehouse, all tables, internal stages, and load all sample data:

```
@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/financial-services/scripts/setup.sql
```

Use EXECUTE IMMEDIATE FROM to run it. After execution, set FINSERV_HOL_WH as the active warehouse for the rest of our session and give me a brief summary of what was created.

1.1 Fallback Prompt

Please read the contents of the file at this Git stage path and execute it as a SQL script, running each statement sequentially:

```
@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/financial-services/scripts/setup.sql
```

Break it into logical sections and show me progress after each section.

1.2 Inspect Prompt

Show me all the tables in FINSERV_HOL_DB.PORTFOLIO and FINSERV_HOL_DB.RISK with their row counts. Display as a clean summary table.

Step 2: Semantic Models

2.1 Primary Prompt

Create a Cortex Analyst semantic view for the portfolio data in FINSERV_HOL_DB.PORTFOLIO.

Include these tables: ADVISORS, CLIENTS, ACCOUNTS, SECURITIES, HOLDINGS, TRANSACTIONS, and BENCHMARKS.

Business intelligence rules:

- Flag "concentrated position" when POSITION_PCT > 15
- Flag "high risk" when RISK_SCORE > 0.8 (from FINSERV_HOL_DB.RISK.RISK_SCORES)

Make sure it can answer questions like:

1. "What is the total AUM by advisor?"
2. "Which accounts have concentrated positions above 15%?"
3. "Show me the top 10 holdings by unrealized P&L"
4. "What is the YTD performance vs S&P 500 benchmark?"

Name it `PORTFOLIO_ANALYTICS` and store it in `FINSERV_HOL_DB.PORTFOLIO`. Register it directly as a semantic view without saving any intermediate files to a stage.

2.1 Fallback Prompt

Copy the pre-built portfolio analytics semantic model from the GitHub repository into the semantic models stage:

Source:

`@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/financial-services/scripts/semantic_models/PORTFOLIO_ANALYTICS_MODEL.yaml`

Destination: `@FINSERV_HOL_DB.PORTFOLIO.SEMANTIC_MODELS_STAGE`

Use `COPY FILES INTO` to transfer it.

2.2 Risk Model

Create a Cortex Analyst semantic view for market risk and compliance analytics.

Include these tables:

- `FINSERV_HOL_DB.RISK.RISK_SCORES`
- `FINSERV_HOL_DB.RISK.COMPLIANCE_ALERTS`
- `FINSERV_HOL_DB.PORTFOLIO.ACCOUNTS`
- `FINSERV_HOL_DB.PORTFOLIO.CLIENTS`

Join `RISK_SCORES` to `ACCOUNTS` on `ACCOUNT_ID`, `COMPLIANCE_ALERTS` to `ACCOUNTS` on `ACCOUNT_ID`, and `ACCOUNTS` to `CLIENTS` on `CLIENT_ID`.

Business intelligence rules:

- Flag "high risk" when `RISK_SCORE > 0.8`
- Flag "critical alert" when `SEVERITY = 'Critical'`
- Flag "open alert" when `STATUS = 'Open'`
- Calculate total Value at Risk (VaR) at 95% confidence from `VAR_95`

Make sure it can answer questions like:

1. "What is the total Value at Risk across all high-risk accounts?"
2. "Show me all open compliance alerts by severity"
3. "Which clients have the highest portfolio risk scores?"
4. "What is the average Sharpe ratio by account type?"

Name it `MARKET_RISK` and store it in `FINSERV_HOL_DB.RISK`. Register it directly as a semantic view without saving any intermediate files to a stage.

2.3 Explore -- Snowsight UI

Guide attendees to Cortex > Analyst. Select `FINSERV_HOL_DB > PORTFOLIO > PORTFOLIO_ANALYTICS`. Open the Playground tab on the right-hand side and ask a question, then click "Add to Verified Queries". Click the Suggestions tab and click "Start Learning". While it learns, explore the other tabs: Custom Instructions, Logical Tables, Relationships, Verified Queries, Monitor.

Step 3: Cortex Search

3.1 Bring PDF into Snowflake

Bring the following PDF from our GitHub repository into FINSERV_HOL_DB.PORTFOLIO so we can make it searchable:

@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/financial-services/pdfs/Risk Management Framework.pdf

3.2 Primary Prompt

Create a Cortex Search service called RISK_DOCS_INFO in FINSERV_HOL_DB.PORTFOLIO that makes the risk management PDF searchable with natural language.

To do this:

- Temporarily scale up FINSERV_HOL_WH for processing power
- Read and parse the PDF content
- Break it into overlapping chunks for better search accuracy
- Index everything so it can be queried in plain English
- Scale the warehouse back down to SMALL when done

3.3 Test Search -- Snowsight UI

Guide attendees to Cortex > Cortex Search. Select FINSERV_HOL_DB (might already be pre-selected). Find RISK_DOCS_INFO and test it.

Step 4: Intelligence Agent

4.1 Primary Prompt

Create a Snowflake Intelligence Agent named FINSERV_AGENT in FINSERV_HOL_DB.PORTFOLIO.

Give it three tools:

1. A data analysis tool named PORTFOLIO_DATA connected to the PORTFOLIO_ANALYTICS semantic model in FINSERV_HOL_DB.PORTFOLIO. It answers questions about clients, accounts, holdings, transactions, and performance. Use FINSERV_HOL_WH.
2. A data analysis tool named RISK_DATA connected to the MARKET_RISK semantic model in FINSERV_HOL_DB.PORTFOLIO. It analyzes risk scores, compliance alerts, and portfolio risk metrics. Use FINSERV_HOL_WH.
3. A document search tool named RISK_DOCS connected to the RISK_DOCS_INFO search service in FINSERV_HOL_DB.PORTFOLIO. It searches risk management documentation for policies and compliance rules.

4.2/4.3 -- Snowsight UI

Navigate to Cortex > Agents > FINSERV_AGENT. Tools are listed on the left-hand side. Ask a couple questions in the chat panel, then open the Monitor tab and click a specific request to see metrics. Click "Preview in Snowflake Intelligence" in the upper-right to switch to the end-user experience. Test the 3 question types: structured data, charting, and document search.

Step 5: Streamlit App

5.1 Chat Page

Build a single-page Streamlit app in Snowflake called FINSERV_ASSISTANT_APP in FINSERV_HOL_DB.PORTFOLIO using FINSERV_HOL_WH.

Use my workspace to create the Streamlit files directly for collaboration.
Use streamlit==1.52.2 for chat features.

Use st.tabs() to create a horizontal tab bar at the top of the page with three tabs: "Chat", "Dashboard", and "Optimization". All three tabs should be visible and switchable at the top -- no sidebar page navigation.

Start with the Chat tab:

- A sidebar with the title "Portfolio Intelligence" and a brief description of the app
- A chat interface where users can have a conversation with FINSERV_AGENT
- Show the agent's responses in the chat as they arrive
- When the agent returns data, display it as a table below the response
- When the agent returns SQL, show it in a collapsible section
- Keep the conversation history visible throughout the session

Apply a premium financial services design throughout the app:

- Main content background: white (#FFFFFF) with light gray panels (#F8FAFC) and subtle card shadows -- clean and professional, not dark
- Sidebar: deep navy (#0F172A) with white text and a gold left border on the active page -- the sidebar is the only dark element
- Primary color: deep navy (#0F172A) for headings and key text; gold (#B45309) as a warm accent for section dividers, active states, and highlights
- Typography: sentence case throughout -- no all-caps; use semibold weight for headers, regular for body; confident and readable, not aggressive
- Data tables: white background, light gray dividers, numbers right-aligned; gains in green (#16A34A), losses in red (#DC2626); clean and data-dense
- KPI cards: white background with a gold top border, large bold number in dark navy (#0F172A), small gray label -- premium but not garish
- Charts: white background with deep navy as the primary series color and gold as the secondary -- consistent with the rest of the app
- Chat input area: white background matching the main content -- seamlessly part of the page, not a floating element
- Overall feel: a premium wealth management portal -- authoritative, polished, and trustworthy; think a private bank client dashboard, not a trading terminal

5.2 Dashboard Page

Fill in the Dashboard tab of FINSERV_ASSISTANT_APP with the following content.

This page should show live data from FINSERV_HOL_DB:

TOP ROW - four summary numbers:

- Total AUM (sum of all account balances)
- Active Clients (count from CLIENTS where STATUS = 'Active')
- Open Compliance Alerts (from RISK.COMPLIANCE_ALERTS where STATUS = 'Open')
- Average Portfolio Risk Score (from RISK.RISK_SCORES)

MIDDLE ROW - two charts side by side:

- A bar chart of AUM by advisor, showing each advisor's total assets under management
- A pie chart of holdings by asset class (Equity, Fixed Income, Alternative)

BOTTOM ROW - a compliance alerts table:

- All open alerts showing: Account Name, Client Name, Alert Type, Severity, Description, Days Open
- Sort by severity (Critical > High > Medium > Low)
- Include a button to download the table as a CSV

INTERACTIVITY:

- Add an advisor filter dropdown at the top that filters all charts and the table
- Add a severity multi-select filter for compliance alerts
- Make the bar chart bars clickable -- clicking an advisor filters the alerts table to show only that advisor's open alerts

Chart styling:

Use plotly for all charts. Color palette: deep navy (#0F172A) as the primary series color, warm gold (#B45309) as the secondary. Use green (#16A34A) for gains/positive and red (#DC2626) for losses/negative. All chart backgrounds should be white (FFFFFF) with light gray grid lines. No dark backgrounds on any chart.

Position the legend below each chart (orientation="h", y=-0.3) to prevent overlap with chart titles. Add a top margin (t=50) on all charts so the title never crowds the plot area.

5.3 Optimization Page

Fill in the Optimization tab of FINSERV_ASSISTANT_APP with the following content.

Title: "Portfolio Intelligence" Subtitle: "Rebalance portfolios and forecast performance using AI and machine learning on your live portfolio data."

At the top of the tab, add a mode selector using radio buttons: - "Rebalancing Optimizer" - "Performance Forecaster"

MODE 1: Rebalancing Optimizer

DATA EXPLORATION (shown immediately when this mode is selected): - An advisor dropdown filter (from ADVISORS table, plus "All advisors") - A risk tolerance multi-select (Conservative / Moderate / Aggressive) - A plotly treemap showing current portfolio allocation by asset class across filtered accounts -- cells colored by concentration level (green within limits, amber approaching, red exceeding 15%) Updates in real-time as filters change. - Below the treemap, a data table of holdings for the filtered accounts (Account, Security, Asset Class, Position %, Market Value, Status)

OPTIMIZATION CONTROLS (below the data exploration): - A slider for Concentration Limit: 5% to 25% (default 15%) - A steel blue "Run Optimizer" button

When clicked: 1. Pulls holdings from HOLDINGS filtered by the selected advisor and risk tolerance, finding positions where POSITION_PCT exceeds the limit 2. Uses linear programming (scipy) to generate sell/buy recommendations that bring each account back within the selected concentration limit while minimizing total notional trade value (fewest trades possible) 3. Respects client risk tolerance: Conservative accounts only receive fixed income buy suggestions, Aggressive accounts allow equity

Display results as: - Three KPI cards: Accounts Needing Rebalancing, Total Notional Trade Value, Estimated Compliance Alerts Resolved - A plotly treemap showing before/after allocation comparison - A recommended sells table: Account, Security, Current %, Target %, Shares to Sell, Estimated Proceeds - A recommended buys table: Security, Asset Class, Shares to Buy, Estimated Cost, Rationale - A button to download recommendations as CSV

MODE 2: Performance Forecaster

DATA EXPLORATION (shown immediately when this mode is selected): - An account dropdown (from ACCOUNTS table) - A date range slider to explore historical data (last 90 days) - A plotly line chart showing historical daily portfolio value from DAILY_MARKET_DATA for the selected account, updating as filters change. Use deep navy for portfolio value. - Below the chart, a summary table of the filtered market data (Date, Portfolio Value, Daily Change %, Cumulative Return %)

FORECASTING CONTROLS (below the data exploration): - Radio buttons for Forecast Horizon: 14 days / 30 days / 60 days - A steel blue "Generate Forecast" button

When clicked: 1. Pull daily portfolio values from DAILY_MARKET_DATA for the selected account 2. Use SNOWFLAKE.ML.FORECAST to train a forecast model on the historical PORTFOLIO_VALUE and predict the next N days (based on the selected horizon). Run this as a SQL operation inside Snowflake. 3. Retrieve forecast values with upper and lower confidence bounds

Display results as: - A headline KPI card: "Forecasted Portfolio Value" at end of horizon, with an arrow and color indicator (green if increasing, red if decreasing) - A plotly line chart combining historical and forecast: - Historical portfolio value (solid navy line, #0F172A) - Forecasted value (dotted line with light gold confidence band) - X-axis: dates, Y-axis: dollar value - Legend below the chart, white background - A risk outlook table: showing periods where the forecast lower bound drops below a key threshold (e.g., 5% below current value), with columns: Date, Forecasted Value, Lower Bound, Potential Drawdown %, Risk Level - A button to download the forecast as CSV

Apply the same styling as the rest of the app throughout: white background, deep navy sidebar (#0F172A), gold accent (#B45309), plotly white chart backgrounds, sentence case, no all-caps.

Add scipy to the app's dependencies. `